



SIGNAL SERIES • PLATFORMS

PLATFORMS & ROADMAPS

Actions, GitLab, ArgoCD — and where the platforms are headed.

By **Nabawy**

AUTHOR & COMPILER

Actions

The Roadmap

The Attack Surface

The Choice

Table of Contents

03	Actions	03	04	CI + CD	04
05	Mechanics	05	06	Versus & Finale	06
07	Part 1: GitLab CI	07	08	Part 1: GitLab CI · continued	08
09	GitLab Review Apps & Links	09	10	GitLab Review Apps & Links	10
11	ArgoCD & GitOps	11	12	ArgoCD & GitOps	12
13	GitOps Delivery	13	14	GitOps Delivery	14
15	The Roadmap	15	16	Product Themes	16
17	Infrastructure & Platform	17	18	Horizon Map	18
19	Roadmap Risks & Handoff	19	20	The Attack Surface	20
21	Pillars A & B	21	22	Pillars A & B · continued	22
23	Pillars C & D	23	24	Platform & Timeline	24
25	Platform Risks & Handoff	25	26	Platform Risks & Handoff	26
27	The Choice	27	28	Side-by-Side	28
29	Decision Framework	29	30B	Tool vs Tool — The Whole Field	30B
30	Convergence	30	31	Decision Risks & Handoff	31
32	Decision Risks & Handoff	32	33B	Kubernetes for Pipelines	33B
33	 Quick Revision — Terms Table	33	34	 Concept Questions — Distilled Answers	34
35	 Command Cheat Sheet	35	36	 Self-Quiz + Answer Key	36

▪ **How to use this book** legend

 NOTE · context  KEY · must retain  WATCH OUT · gotcha  VERIFIED · check

NOTE Read cover to cover once, then use the Contents to jump. Every chapter ends with a  EXAM box — if you can answer it cold, move on.

Actions

Picking up from *Troubleshooting*, where the self-hosted domain concluded — here is *the same loop with zero infrastructure*.

01 / CONCEPT MAP

Generic	Actions
Pipeline	Workflow (.yml)
Job / Step	jobs.<id> / run: or uses:
Runner	Hosted (ubuntu-latest) or self-hosted
Trigger	on: push, PR, schedule, dispatch
Gate	environment: + required reviewers

Threads: [Controllers & Agents](#) · [Approval Gates](#)

WHEN TO USE GITHUB ACTIONS use when source already lives on GitHub · use when zero-infra SaaS runners fit · use when Marketplace covers the task · avoid when heavy self-hosted compliance fleets · avoid when source lives outside GitHub. **Tool vs Tool — The Whole Field**

CI + CD

02 / CI IN ONE FILE

```
on: [pull_request, push→main]
jobs:
  build → test → scan (needs: build, parallel)
  checkout@v4 + setup-node + cache on every job
```

03 / CD WITH GATES

```
deploy-staging:    needs [test, scan] # auto
  environment: staging
deploy-production: needs staging    # gated
  environment: production # required reviewers = approval
```

Threads: [Approval Gates](#)

Mechanics

04 / RUNNERS & COST

- **Labels select runners** — wrong labels = queue forever.
- **Trust** — fork PRs never touch persistent self-hosted runners.
- **Cost** — public free; private pool 2–3k min/mo; then ~\$0.006 Linux / \$0.010 Windows / \$0.062 macOS. macOS burns 10×.
- **Killers** — macOS builds, no cache, sprawling matrices, failed jobs bill like green ones.

05 / POWER PATTERNS

REUSABLE (workflow_call) — one pipeline, N repos. **MATRIX** — fan out versions in one block. **CONCURRENCY** — cancel stale runs. **PATHS** — docs-only PRs skip CI. **SETUP** — drop the YAML, push, watch Actions tab, require the check in branch protection.

ARC SCALE SET ephemeral runners on your cluster, scale-to-zero idle, burst to max on PR storms. Remote caches (not pod disks). Wins when hosted minutes cost more than cluster ops.

Threads: [Controllers & Agents](#)

Versus & Finale

06 / ACTIONS VS JENKINS

	Actions	Jenkins
Runs	GitHub cloud, fresh VM	Your servers
Language	YAML in repo	Groovy Jenkinsfile
Cost	Free pool, then meter	Free license + your labor

One line: Actions trades control for convenience; Jenkins trades convenience for control.

Q1 needs: vs parallel?

Jobs run parallel by default; needs : sequences them into gates.

07 / HANDOFF

Both implementations behind you — the full arc from commit to production is yours. Return to the Topic Index for deployment strategies and labs, or start building.

 **EXAM** Actions in one breath: the pipeline is a **workflow YAML** where jobs.<id> run run:/uses: steps on hosted (ubuntu-latest) or self-hosted runners, fired by on: push, PR, schedule or dispatch; jobs run **parallel by default** and needs : sequences them into gates, environment: plus required reviewers gates production, and workflow_call, **matrix**, **concurrency** and **paths** keep fleets cheap — one line: Actions trades control for convenience where Jenkins trades convenience for control.

 Next: [GitLab CI & GitOps](#)

Threads: [Controllers & Agents](#)

Part 1: GitLab CI

PART 1: GITLAB CI

GitLab CI is the pipeline engine inside GitLab: one YAML file in the repo (`.gitLab-ci.yml`) driving stages, runners, and environments — the same concepts as Jenkins and Actions, with GitLab's own vocabulary.

1. WHAT IS IT?

Picking up from **GitHub Actions**, where workflows lived beside the code — GitLab goes further: repo, pipeline, registry, and environments are one product, so the pipeline natively knows about merge requests, environments, and review apps.

2. PIPELINE ANATOMY (GITLAB VOCABULARY)

```
stages:
  - build
  - test
  - deploy

build:
  stage: build
  script:
    - echo "Building..."

test:
  stage: test
  script:
    - echo "Testing..."

deploy:
  stage: deploy
  script:
    - echo "Deploying..."

needs: [build] # DAG edge -- runs as soon as build passes
```

- **Stages** run in order; **jobs** in the same stage run in parallel.
- `needs:` builds a DAG (like Actions), skipping stage waits.
- `rules:` decides when jobs run (merge request only, `main` only, paths changed) — replaces the older `only/except`.
- `artifacts:` passes files between jobs (reports, coverage, JUnit); `cache:` speeds reruns (lockfile-keyed, same discipline as everywhere).

3. RUNNERS & TAGS

Runner scope	Use
Shared	Fleet managed by the instance; default for most jobs
Group / project (specific)	GPU, private network, compliance isolation
Tags (tags: [gpu])	Route jobs to capable runners

Threads: [Controllers & Agents](#) · [Artifact Digest](#) · [Hermetic Builds](#)

WHEN TO USE GITLAB CI use when repo, pipeline and envs stay one product · use when review apps per merge request help · use when merge trains serialize main · avoid when polyglot multi-platform estates · avoid when pull-model K8s delivery dominates. **Tool vs Tool — The Whole Field**

Part 1: GitLab CI • continued

One job = one runner by default; scale = more runners. Self-managed runners need the same patching/scaling discipline as any self-hosted fleet.

4. ENVIRONMENTS, REVIEW APPS & MERGE TRAINS

- **Environments** (`environment: staging/production`) track what's deployed where; production gates add manual approvals — the same delivery-vs-deployment line as everywhere.
- **Review apps** (`on_stop: teardown`) spin a live environment per merge request and destroy it on merge — GitLab's built-in preview environments.
- **Merge trains** serialize merges onto latest green `main` with re-verification — the same race the merge queue solves elsewhere.

5. GITLAB VS THE SISTERS

GitLab CI	GitHub Actions	Jenkins	
Config lives	.gitlab-ci.yml in repo	.github/workflows/	Jenkinsfile in repo
Runners	Shared fleet + tags	Hosted + ARC	Controller + agents
Environments	Built-in (+ review apps)	Environments as gates	Manual promotion jobs
Best when	Already on GitLab; want one product	Already on GitHub	Need total control

6. FAILURE MODES

- Shared-runner queues at peak → pin critical jobs to tagged runners or add project runners.
- `rules:` mis-scoping → jobs silently skip; every `rules:` change deserves a pipeline dry-run on a test MR.
- Artifacts bloating storage → expire aggressively (`expire_in:`), keep images in the registry not in job artifacts.

7. INTERVIEW NOTES

- `needs:` vs stage ordering: when the DAG matters.
- Review apps lifecycle: create on MR, destroy on merge (`on_stop`).
- Merge trains: which race they eliminate.

Threads: [Controllers & Agents](#) • [Artifact Digest](#) • [Approval Gates](#)

GitLab Review Apps & Links

RELATED TOPICS

- GitHub Actions
- Continuous Integration
- Environments & Release

Pipelines push changes — something has to keep the cluster matching git. Continue in **ArgoCD & GitOps**.

PART 2: ARGOCD & GITOPS

GitOps: git is the desired state, an in-cluster operator pulls reality toward it. ArgoCD is that operator for Kubernetes — the pipeline stops pushing to prod and starts committing to git.

1. WHAT IS IT?

Picking up from **GitLab CI**, where pipelines pushed artifacts outward — ArgoCD inverts the last mile: the CI pipeline builds the image and commits the new digest to a manifests repo; ArgoCD (running inside the cluster) notices and syncs. Cluster credentials never leave the cluster.

2. PUSH VS PULL

Push (classic CD)	Pull (GitOps)	
Actor	External pipeline holds cluster creds	In-cluster agent, creds stay inside
Source of truth	Pipeline run history	Git commit
Rollback	Redeploy old digest	git revert
Drift	Silent until noticed	Detected + optionally auto-healed

Threads: [GitOps Pull Model](#) · [Artifact Digest](#) · [Rollback & Recovery](#)

GitLab Review Apps & Links • continued

3. CORE OBJECTS

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: checkout
spec:
  source: { repoURL: https://git/ops, targetRevision: main, path: apps/checkout
  destination: { server: https://kubernetes.default.svc, namespace: prod
  syncPolicy: { automated: { prune: true, selfHeal: true } }
```

- **Application:** what (repo+path+revision) goes where (cluster+namespace).
- **Sync waves + hooks:** order resources (CRDs before CRs, migrations before app) — the deployment strategy's sequencing, declaratively.
- **ApplicationSet:** one template → N environments/tenants; preview environments become a generator, not a script.
- **Image Updater (optional):** tracks new digests and commits them — automation with git as the audit trail.

4. WHERE IT FITS

CI still builds, tests, scans, signs (everything upstream unchanged). The handoff changes: instead of `kubectl apply` from CI, CI commits `image: sha-9f3a` to the ops repo. Promotion = pull request between env branches/folders — reviewable, revertable, auditable.

5. FAILURE MODES

- Auto-sync + bad commit → bad state propagates at machine speed; gate with manual sync on prod or automated kill criteria.
- Secrets in the manifests repo → use external secret operators (External Secrets, Sealed Secrets); git holds references, never values.
- `selfHeal` fighting hotfixes → emergency `kubectl` edits get reverted; declare a break-glass procedure (pause sync, fix, commit).
- Monorepo scale → thousands of Applications; shard by team/cluster with ApplicationSets.

6. INTERVIEW NOTES

- Push vs pull: credential blast radius is the core argument.
- What sync waves solve (ordering without scripts).
- Rollback in GitOps: why `git revert` beats redeploy.

Threads: [GitOps Pull Model](#) • [Rollback & Recovery](#) • [Artifact Digest](#)

RELATED TOPICS

- Continuous Delivery
- Deployment Strategies
- Artifact Management
- GitLab CI

GitOps declares app state — but environments themselves are infrastructure. Continue in **laC for Environments**.

ArgoCD & GitOps

GitLab CI is the pipeline engine inside GitLab: one YAML file in the repo (`.gitLab-ci.yml`) driving stages, runners, and environments — the same concepts as Jenkins and Actions, with GitLab's own vocabulary.

1. WHAT IS IT?

Picking up from **GitHub Actions**, where workflows lived beside the code — GitLab goes further: repo, pipeline, registry, and environments are one product, so the pipeline natively knows about merge requests, environments, and review apps.

2. PIPELINE ANATOMY (GITLAB VOCABULARY)

```
stages: [build, test, deploy]
build:
  stage: build
  script: [npm ci, npm run build]
test:
  stage: test
  script: [npm run test]
```

- **Stages** run in order; **jobs** in the same stage run in parallel.
- **needs:** builds a DAG (like Actions), skipping stage waits.
- **rules:** decides when jobs run (merge request only, `main` only, paths changed) — replaces the older `only/except`.
- **artifacts:** passes files between jobs (reports, coverage, JUnit); **cache:** speeds reruns (lockfile-keyed, same discipline as everywhere).

3. RUNNERS & TAGS

Runner scope	Use
Shared	Fleet managed by the instance; default for most jobs
Group / project (specific)	GPU, private network, compliance isolation
Tags (tags: [gpu])	Route jobs to capable runners

One job = one runner by default; scale = more runners. Self-managed runners need the same patching/scaling discipline as any self-hosted fleet.

4. ENVIRONMENTS, REVIEW APPS & MERGE TRAINS

Threads: [Controllers & Agents](#) · [Artifact Digest](#) · [Hermetic Builds](#)

WHEN TO USE ARGOCD use when Kubernetes fleets need pull delivery · use when git audit trail is mandatory · use when env-per-branch scale grows · avoid when non-Kubernetes estates · avoid when single-service shops. **Tool vs Tool — The Whole Field**

ArgoCD & GitOps • continued

- **Environments** (`environment: staging/production`) track what's deployed where; production gates add manual approvals — the same delivery-vs-deployment line as everywhere.
- **Review apps** (`on_stop: teardown`) spin a live environment per merge request and destroy it on merge — GitLab's built-in preview environments.
- **Merge trains** serialize merges onto latest green `main` with re-verification — the same race the merge queue solves elsewhere.

5. GITLAB VS THE SISTERS

GitLab CI	GitHub Actions	Jenkins	
Config lives	<code>.gitlab-ci.yml</code> in repo	<code>.github/workflows/</code>	Jenkinsfile in repo
Runners	Shared fleet + tags	Hosted + ARC	Controller + agents
Environments	Built-in (+ review apps)	Environments as gates	Manual promotion jobs
Best when	Already on GitLab; want one product	Already on GitHub	Need total control

6. FAILURE MODES

- Shared-runner queues at peak → pin critical jobs to tagged runners or add project runners.
- `rules:` mis-scoping → jobs silently skip; every `rules:` change deserves a pipeline dry-run on a test MR.
- Artifacts bloating storage → expire aggressively (`expire_in:`), keep images in the registry not in job artifacts.

7. INTERVIEW NOTES

- `needs:` vs stage ordering: when the DAG matters.
- Review apps lifecycle: create on MR, destroy on merge (`on_stop`).
- Merge trains: which race they eliminate.

RELATED TOPICS

- GitHub Actions
- Continuous Integration
- Environments & Release

Pipelines push changes — something has to keep the cluster matching git. Continue in **ArgoCD & GitOps**.

Threads: [Controllers & Agents](#) • [Artifact Digest](#) • [Approval Gates](#)

GitOps Delivery

GitOps: git is the desired state, an in-cluster operator pulls reality toward it. ArgoCD is that operator for Kubernetes — the pipeline stops pushing to prod and starts committing to git.

1. WHAT IS IT?

Picking up from **GitLab CI**, where pipelines pushed artifacts outward — ArgoCD inverts the last mile: the CI pipeline builds the image and commits the new digest to a manifests repo; ArgoCD (running inside the cluster) notices and syncs. Cluster credentials never leave the cluster.

2. PUSH VS PULL

Push (classic CD)	Pull (GitOps)	
Actor	External pipeline holds cluster creds	In-cluster agent, creds stay inside
Source of truth	Pipeline run history	Git commit
Rollback	Redeploy old digest	git revert
Drift	Silent until noticed	Detected + optionally auto-healed

3. CORE OBJECTS

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: checkout }
spec:
  source: { repoURL: https://git/ops, targetRevision: main, path: apps/checkout }
  destination: { server: https://kubernetes.default.svc, namespace: prod

```

- **Application:** what (repo+path+revision) goes where (cluster+namespace).
- **Sync waves + hooks:** order resources (CRDs before CRs, migrations before app) — the deployment strategy's sequencing, declaratively.
- **ApplicationSet:** one template → N environments/tenants; preview environments become a generator, not a script.
- **Image Updater (optional):** tracks new digests and commits them — automation with git as the audit trail.

4. WHERE IT FITS

Threads: [GitOps Pull Model](#) · [Artifact Digest](#) · [Rollback & Recovery](#)

GitOps Delivery • continued

CI still builds, tests, scans, signs (everything upstream unchanged). The handoff changes: instead of `kubectl apply` from CI, CI commits `image: sha-9f3a` to the ops repo. Promotion = pull request between env branches/folders — reviewable, revertable, auditable.

5. FAILURE MODES

- Auto-sync + bad commit → bad state propagates at machine speed; gate with manual sync on prod or automated kill criteria.
- Secrets in the manifests repo → use external secret operators (External Secrets, Sealed Secrets); git holds references, never values.
- `selfHeal` fighting hotfixes → emergency `kubectl` edits get reverted; declare a break-glass procedure (pause sync, fix, commit).
- Monorepo scale → thousands of Applications; shard by team/cluster with `ApplicationSets`.

6. INTERVIEW NOTES

- Push vs pull: credential blast radius is the core argument.
- What sync waves solve (ordering without scripts).
- Rollback in GitOps: why `git revert` beats redeploy.

RELATED TOPICS

- Continuous Delivery
- Deployment Strategies
- Artifact Management
- GitLab CI

GitOps declares app state — but environments themselves are infrastructure. Continue in **laC for Environments**.

 **EXAM** GitLab plus GitOps in one breath: **GitLab CI** drives `.gitlab-ci.yml` stages with `needs`: DAG edges, `rules`: gating, `artifacts:/cache`: passing, shared or tagged runners, built-in environments with **review apps** (`on_stop`: teardown) and **merge trains** killing the green-branch-red-main race — then **ArgoCD inverts the last mile**: CI commits `image: sha-9f3a` to the ops repo and the in-cluster **Application** (source + destination + `syncPolicy`) pulls, orders via **sync waves**, scales via **ApplicationSets**, rolls back with `git revert`, and never stores secret values in git.

▶ Next: [The Roadmap](#)

Threads: [GitOps Pull Model](#) • [Rollback & Recovery](#) • [Artifact Digest](#)

The Roadmap

Jenkins ships on *cadence, not calendar* — every week a new release, every four weeks a stabilized LTS, security fixes whenever they are needed. The 2026 roadmap is organized into *horizons*, not hard dates, because contributor bandwidth decides when an initiative ships.

01 / RELEASE CADENCE — CURRENT MODEL

Unlike vendor-driven products, Jenkins never announces a "Jenkins 3.0." It evolves as a rolling train of weekly releases; the LTS line is a curated snapshot for conservative upgraders.

Track	Rhythm	Who should follow
Weekly	Every week (2.545 → 2.566+ in 2026)	Teams that want features fastest; accept more churn
LTS	Cut every ~4 weeks + backports	Most production controllers — stable, patched
Security	As needed via jenkinsci-advisories	Everyone — apply out-of-band when CVE drops

2026 SHIPPED 2.545 (Jan) Java 21+ required on controller · 2.552 (Feb) experimental Plugin Manager UI + revamped Clouds page · 2.554 (Mar) script console adapted for new UI · 2.556 (Mar) Spring Security 7 + Spring Framework 7 · LTS 2.541.1 (Jan) unified RPM repo, new GPG key for RPM/DEB.

02 / HOW HORIZONS WORK

RELEASED

PREVIEW

CURRENT

NEAR TERM

FUTURE

Released is done. **Preview** is usable but still changing. **Current** is under active development. **Near Term** is next up once bandwidth allows. **Future** has consensus but no active work. Treat Near/Future as directional priorities — the Jenkins project explicitly warns they are not committed dates.

READING RULE
Jenkins does not commit to delivery dates — all initiatives depend on volunteer/contributor availability. Track progress via the live `roadmap.yml` on GitHub, not via this snapshot.

Product Themes

03 / PIPELINE AUTHORIZING & DEVELOPER TOOLING

Initiative	Horizon	What changes for you
Pipeline as YAML PREVIEW	Preview	Define Pipelines in YAML natively — no extra shared-library shim. Simpler for teams that prefer YAML over Groovy.
Jenkins Templating Engine 2.0 RELEASED	Released + iterating	Tool-agnostic pipeline templates; reuse pipeline logic across repos.
IDE pipeline dev (debug, visual editor) FUTURE	Future	No active dev yet — design intent only. Wiring live debug + visual editors into IDE plugins.
Expanded Pipeline reference docs FUTURE	Future	Completeness pass on DSL/step docs.

04 / END-USER EXPERIENCE & INTERFACE

- **Plugin Manager UI revamp** — Preview → shipping incrementally. Experimental UI landed in 2.552; full rollout is staged across weeklies. Expect iterative polish, not a flag day.
- **Look & Feel updates** — Current. Systematic styling modernization across the web UI.
- **Accessibility** — Near Term. A11y pass after the style layer stabilizes.
- **Full interface rework** — Future. Larger overhaul; community consensus exists, no active implementation yet.
- **Already released:** Dark Theme, Configuration UI (tables → divs), Theme Manager plugin.

WHAT THIS MEANS IN PRACTICE

If you maintain internal Jenkins docs/screenshots, budget for UI drift through 2026. The Plugin Manager and Clouds pages have already changed shape — automation (Puppet/JCasC) that screen-scrapes the UI will need updates.

05 / MANAGEMENT & TOOL INTEGRATIONS

Area	Item	Horizon
Integrations	GitHub Checks API + GitHub App auth	RELEASED
Integrations	OpenAPI spec for core + plugins (client generation)	FUTURE
Admin	Script Security — granular approve/deny/revoke (replaces all-or-nothing)	CURRENT
Admin	Windows Services: YAML config support	CURRENT
Admin	JCasC: pluggable external config (Git, S3, K8s CRDs)	FUTURE
Admin	Better Remoting monitoring (Prometheus/Grafana for controller↔agent)	FUTURE

Threads: [Signals & SLOs](#) · [Controllers & Agents](#)

Infrastructure & Platform

06 / JENKINS ON CLOUD PLATFORMS

Initiative	Horizon	Intent
CloudEvents support CURRENT	Current	Standard eventing for interoperability (Tekton, Knative, external notifiers).
Jenkinsfile Runner 1.0 · Tekton build steps · Pluggable log/storage PREVIEW	Preview	Run Jenkinsfiles without a full controller; store logs/results externally.
Tekton as alt execution engine · FaaS capability · Fully pluggable storage FUTURE	Future	Longer bets on alternate runtimes and decoupled storage.

07 / PACKAGING & PLATFORM SUPPORT

SHIPPED Eclipse Temurin JDK migration, Windows installer rework, multi-arch Docker images. **FUTURE** — Cloud-native Java (GraalVM/Quarkus) for smaller/faster images. **DEV TOOLING SHIPPED** — JEP-229 (Continuous Delivery of Plugins), plugin BOMs, automated dependency management. **PREVIEW** — Expanded static analysis for security (Find Security Bugs coverage).

08 / PROJECT INTERNALS — INFRA TEAM 2026 FOCUS

Tracked weekly by the Infra team. Small team, real budget (~\$5–6k/mo Azure sponsorship), steady burn-down:

 **PUPPET → ANSIBLE** Current, in progress — Puppet flagged strategic risk (Feb 2025)

↓

 **WINDOWS 2025 + JDK 25** Near term — deprecate Win2019, switch infra to JDK 25

↓

 **SIGN MSI WITH LF CERT** Top priority — DigiCert expiry May 2026; LF cert replacement

↓

 **STATS + ACCOUNTS MODERNIZATION** Resume stats.jenkins.io; modernize accounts.jenkins.io (LDAP) — future

- Ongoing: Terraform/Helm/jq bumps, Playwright browser packaging, credential rotation, budget stewardship.
- Governance:** JEP-14 (public roadmap process) + JEP-15 (crowdfunding) — **RELEASED**; **Technical Steering Committee** — **FUTURE**.

Threads: [Controllers & Agents](#)

Horizon Map

09 / FULL HORIZON TABLE (SEPTEMBER 2026 SNAPSHOT)

Theme	Item	Horizon
Pipeline	Pipeline as YAML	PREVIEW
Pipeline	JTE 2.0	RELEASED
Pipeline	IDE debugging / visual editor	FUTURE
UX	Plugin Manager UI revamp	PREVIEW
UX	Look & Feel updates	CURRENT
UX	Accessibility	NEAR TERM
Admin	Script Security granular approvals	CURRENT
Admin	JCasc pluggable config sources	FUTURE
Cloud	CloudEvents	CURRENT
Cloud	Jenkinsfile Runner / Tekton steps	PREVIEW
Infra	Puppet → Ansible	CURRENT
Infra	JDK 25 + Win2025 + MSI LF cert	NEAR TERM
Gov	Technical Steering Committee	FUTURE

10 / COMMUNITY & PROGRAM HIGHLIGHTS — 2026

- **Google Summer of Code 2026** — Jenkins as mentoring org; ~300+ proposals, 8 ideas, focus on **docs site retooling** (migration to Docusaurus incl. Chinese site) and **Contributor Spotlight / Success Stories** retooling (Gatsby → Vite).
- **Jenkins Awards 2026** — nominations open: Most Valuable Contributor, Security MVP, Most Valuable Advocate.
- Continuous **security advisories** (e.g., stored XSS in node offline-cause descriptions; plugin CVEs).

11 / HOW TO TRACK PROGRESS

```
# live data – always verify against the repo, not this PDF
roadmap.yml → github.com/jenkins-infra/jenkins.io/content/_data/roadmap/roadmap.yml
archive.yml → ../roadmap/archive.yml # completed / withdrawn
JEP-14 → github.com/jenkinsci/jep/tree/master/jep/14 # roadmap process
infra notes → community.jenkins.io # Infra team meeting threads (weekly)
# suggest an item via JEP-14 – anyone can propose
```

Threads: [Approval Gates](#) · [Controllers & Agents](#)

Roadmap Risks & Handoff

12 / FAILURE MODES — ROADMAP EDITION

TREATING NEAR TERM AS A COMMITMENT

The most common misread. Near/Future are priorities ordered by contributor interest, not sprint commitments. A "Near Term" label is not a quarter promise — it means "next up when someone can staff it." Plan adoption as *if/when*, not *when*.

- **Pinning automation to weekly UI screenshots** — UI/UX is actively churning (Plugin Manager, Look & Feel). Gate UI automation behind CSS selectors and cache-busting, or prefer JCasC/Groovy init over scraping.
- **Ignoring the cert window** — MSI DigiCert expiry May 2026 is the infra team's top priority; Windows install flows that verify publisher certs can break if you lag the migration.
- **Staying on Puppet assumptions** — Puppet → Ansible is in progress. If you contribute infra or mirror infra patterns internally, expect playbook drift.
- **Assuming "Pipeline as YAML" replaces Groovy tomorrow** — it's Preview. Groovy Jenkinsfile and JTE remain the production path; YAML is additive.

13 / OPERATOR CHECKLIST

ADOPT LTS line + advisories; track `roadmap.yml` weekly if you depend on Preview items. **PLAN** — Java 21 now mandatory; qualify JDK 25 early; Windows Server 2025 qualification path. **HARDEN** — Script Security granular approvals available now — move off blanket approvals. **PREP** — CloudEvents and pluggable log/storage for Tekton-interop experiments.

Q2 Should we wait for the "full UI rework" before upgrading Jenkins?

No. That rework has consensus but no active development. Upgrade on the LTS cadence; the incremental UI improvements (dark theme, Plugin Manager) ship steadily regardless.

14 / HANDOFF

Jenkins is moderning its *house* — UX, plugin trust, and infra debt — not reinventing the pipeline. Next, see **GitHub Actions Roadmap 2026** for the platform optimizing hardest for supply-chain trust, then **the Decision Guide** for the decision frame that holds both roadmaps side by side.

📌 **EXAM** The Jenkins roadmap in one breath: Jenkins ships on **cadence, not calendar** — weekly releases, LTS every ~4 weeks, security advisories as needed — with 2026 highlights Java 21+ (2.545), Plugin Manager UI preview (2.552) and Spring 7 (2.556), organized into **horizons** (Released, Preview, Current, Near Term, Future) that are **priorities, not quarters**; infra focus is Puppet → Ansible, JDK 25 plus Windows 2025, and the May 2026 MSI signing cert — track the live `roadmap.yml`, never this snapshot.

▶ Next: [The Attack Surface](#)

Threads: [Approval Gates](#)

The Attack Surface

In 2025–26 a pattern emerged: *compromise a trusted Action* → *execute inside CI* → *exfiltrate secrets*. The roadmap is GitHub's direct answer to three root causes: *implicit trust in workflow dependencies*, *poor execution visibility*, and *overly broad credential exposure*.

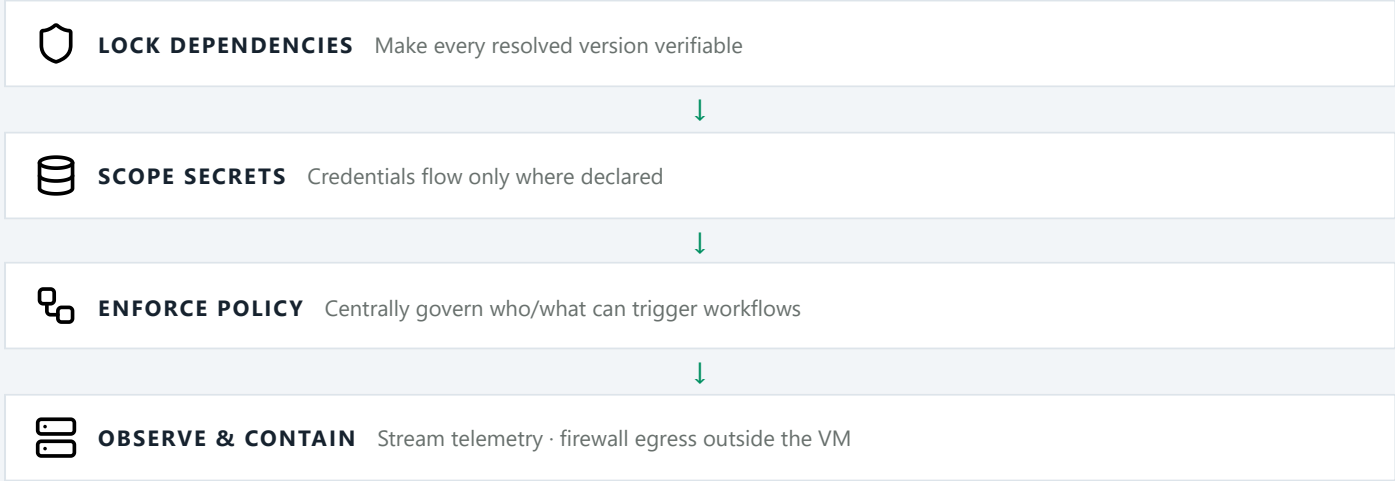
01 / INCIDENTS THAT DROVE THE ROADMAP

Incident	Technique	What it abused
tj-actions/changed-files	Compromised trusted Action	Unpinned uses : + transitive trust
Nx build-tool compromise	Supply-chain injection	Shared toolchain dependency
trivy-action tag poisoning	75 tags force-pushed to malicious commits	Mutable tags / releases
"Clinejection"	Prompt injection via issue title → cache poisoning → exfil	Untrusted event input driving agent workflows

THE PATTERN

Every incident follows the same pipeline: a normal-looking workflow pulls an Action at a friendly tag, that Action (or its transitive dependency) is silently retargeted, and secrets flow outward before anyone notices. SHA pinning *at the top level only* does not close this gap.

02 / ROADMAP PILLARS



Threads: [Hermetic Builds](#) · [Controllers & Agents](#)

Pillars A & B

03 / A — LOCKING WORKFLOW DEPENDENCIES

A new `dependencies:` section in workflow YAML. Works like `go.sum` / `Cargo.lock` — but for Actions. GitHub will resolve and lock **direct and transitive** Action dependencies via commit SHA + cryptographic hash verification.

```
name: ci
on: pull_request
dependencies: # NEW — lockfile for Actions
actions/checkout: sha256:abc123... # SHA + hash, verified
my-org/composite-action: sha256:def456... # transitive deps locked too
jobs:
build:
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@abc123 # deterministic, diffable
```

WHY THIS MATTERS Today's SHA pinning only covers top-level `uses:` lines, not what a composite Action calls internally. Locking closes that gap: **deterministic execution**, **visible dependency diffs in PRs**, and **tamper detection even when a SHA looks correct** (hash mismatch fails the run). Paired with **immutable releases** and stricter publishing rules for maintainers.

	Before	After (locking)
Scope	Top-level <code>uses:</code> only	Transitive closure (composite Actions included)
Verification	SHA equality	SHA + cryptographic hash
PR visibility	No dependency diff	Lockfile diff surfaces version drift

Pillars A & B • continued

04 / B — SCOPED SECRETS

Secrets will be bindable to a narrower scope instead of flowing broadly by default — and will no longer auto-inherit into reusable workflows.

Scope	Secret is available only when
Repository	Workflow runs inside that specific repo
Branch	Workflow runs targeting that branch
Environment	Job targets the environment (e.g., production)
Trusted workflow	Only an explicitly approved reusable workflow calls it

```
# BEFORE — secrets auto-inherit
jobs:
  deploy:
    uses: org/central-deploy@v2
    secrets: inherit          # ← broad, implicit
# AFTER — explicit per-secret binding
jobs:
  deploy:
    uses: org/central-deploy@v2
    secrets:
      NPM_TOKEN: ${ secrets.NPM_TOKEN } # scoped to trusted workflow + env
```

PERMISSION CHANGE Repository write access will no longer automatically grant secret-management rights. That moves to custom/delegated roles (separation of duties). Treat `secrets: inherit` as deprecated — audit for it now.

Threads: [Controllers & Agents](#)

Pillars C & D

05 / C — POLICY-DRIVEN WORKFLOW EXECUTION PROTECTIONS

Extends the existing **rulesets** framework to centrally govern workflow execution — replacing scattered `if: github.actor != 'dependabot[bot]'` guards spread across individual workflow files.

Rule type	Purpose	Example
Actor rules	Who/what can trigger workflows	Only humans, Dependabot, or a specific GitHub App can fire <code>deploy.yml</code>
Event rules	Which events can trigger runs	<code>push</code> allowed; <code>issue_comment</code> blocked unless from maintainer

DIRECTLY TARGETS "COMMENT-AND-CONTROL"
Untrusted contributors trigger agent/automation workflows via issue or PR comments, injecting instructions into AI-assisted runs. Central policy prevents the trigger from ever reaching the runner — no per-workflow patching needed. **Status:** Public preview shipped **June 18, 2026**.

```
# org-level ruleset (conceptual)
ruleset: workflow-execution
  target: workflows
  conditions:
actor: [human, dependabot[bot], my-deploy-app]
  event: [push, pull_request, workflow_dispatch]
# issue_comment / discussion events require actor allow-list
```

06 / D — ENDPOINT MONITORING & CONTROL

Actions data stream — near-real-time telemetry for workflow runs, streamed to Amazon S3 or Azure Event Hub / Data Explorer. Designed for SIEM/SOC consumption.

- Workflow execution metadata — status, duration, runner type
- Dependency-resolution detail — which Action resolved to which version
- Network activity — planned; feeds the firewall policy workflow

Native egress firewall (Layer 7) — runs *outside* the GitHub-hosted runner VM, so code inside the runner cannot bypass it.

Mode	Behavior	Use when
Monitoring	Logs all outbound traffic	Building your allow-list without breakage
Enforcement	Blocks requests outside the allow-list	After allow-list is validated in monitoring

ROLLOUT DESIGN Two-mode rollout is intentional: run in Monitoring to discover destinations, cut to Enforcement only when vetted. This is how GitHub avoids "enforcement day breaks every CI" — mirror the same staged adoption inside your org.

Threads: [Controllers & Agents](#)

Platform & Timeline

07 / PLATFORM & RUNNER IMPROVEMENTS — SHIPPED 2026

Capability	Status	Why it matters
Arm64 Linux + Windows runners (GitHub-hosted)	GA — Sept 2026	Performance/cost/sustainability (real cases: 30 min x86 → ~4 min Arm64); Team + Enterprise Cloud
Arm64 labels ubuntu-22.04-arm, ubuntu-24.04-arm, windows-11-arm	GA incl. private repos	Native ARM without virtualization; public repos free
Immutable Releases	GA — all orgs/repos	Closes tag-mutability gap (the trivy-action attack vector)
Build provenance / SBOM attestations (actions/attest-build-provenance)	Maturing as standard	SLSA-style provenance for release hardening

IMMUTABLE RELEASES – THE QUIET FIX

Git tags (and therefore Releases) were historically mutable — an attacker could silently retarget a tag that workflows pinned. Immutable Releases makes that retarget detectable/blocked. Enable it at the org level; pair with dependency locking for end-to-end provenance.

08 / ADJACENT TREND — AI / AGENTIC WORKFLOWS

Copilot coding agent and custom agentic workflows now trigger directly from Actions (PR review agents, release-notes automation, "agent-of-day" patterns). This trend is *driving* the security roadmap — agent-triggered workflows responding to `issue` / `pull_request` / `discussion` events are exactly the attack surface that Workflow Execution Protections + scoped secrets are designed to contain. Community tooling (e.g., `gh-aw`) is fast-moving but outside GitHub's core roadmap.

09 / SUMMARY TIMELINE (GITHUB'S ANNOUNCED WINDOWS)

Feature	Public Preview	GA
Workflow dependency locking	3–6 months from announcement	+6 months after preview
Scoped secrets	3–6 months	+6 months
Secret permission model (delegated roles)	3–6 months	TBD
Workflow Execution Protections (rulesets)	Shipped — Jun 18, 2026	TBD
Actions data stream (S3/Event Hub)	3–6 months	+6–9 months
Native egress firewall (L7)	6–9 months	TBD

Timelines are GitHub's "X to Y months" estimates as of the initial roadmap announcement (Feb–Mar 2026). GitHub does not publish hard SLAs — re-verify via the Changelog.

Threads: [Artifact Digest](#) · [Supply-Chain Security](#) · [Controllers & Agents](#)

Platform Risks & Handoff

10 / ARCHITECTURE — HOW THE PILLARS COMPOSE



WORKFLOW YAML + DEPENDENCIES LOCKFILE

Every Action pinned to SHA+hash — transitive closure verified at queue time



RULESET GATE — ACTOR + EVENT

Org ruleset decides: can this actor fire this event on this workflow?



SCOPED SECRET INJECTION

Only branch/env/trusted-workflow-scoped secrets enter the job



RUNNER (OUT-OF-VM FIREWALL)

L7 egress enforcement outside the VM · streams telemetry to S3/Event Hub



ARTIFACT + ATTESTATION

Immutable release · SLSA provenance · SBOM

11 / FAILURE MODES

- **Enforcing the firewall without a monitoring phase** — you will deny legitimate package registries and prod endpoints. Always run Monitoring → validate allow-list → Enforcement.
- **Keeping secrets: inherit** — broad inheritance defeats scoped secrets. Grep org for inherit and convert to per-secret passing.
- **Pinning top-level only** — transitive Actions remain unpinned until locking ships GA. Until then, vendor composite Actions or audit their inner uses:.
- **Tag-pinning without Immutable Releases** — a tag can still be retargeted. Immutable Releases + SHA+hash locking are the pair.
- **Leaving issue_comment triggers unrestricted** — with agentic workflows, this is a prompt-injection channel. Gate it with Workflow Execution Protections immediately (already in preview).

Q3 Which pillar should we adopt first?

Workflow Execution Protections (already in preview) + enable Immutable Releases today. Scoped secrets and dependency locking in preview next — enroll early. Firewall last, after a monitoring window.

Threads: [Artifact Digest](#) · [Supply-Chain Security](#) · [Controllers & Agents](#)

Platform Risks & Handoff · continued

12 / HOW TO TRACK PROGRESS

```
# verify timelines – roadmaps are directional, Changelog is ground truth
roadmap → github.com/github/roadmap # filter label:actions; stages: planned/beta/GA
changelog → github.blog/changelog # actual ship dates: Arm64 GA, Immutable GA, etc.
feedback → GitHub Public Feedback Discussions # input on roadmap items
```

WHEN TO ADOPT

Preview features are for non-production evaluation and telemetry pipelines; gate production rollouts on GA + your own egress allow-list validation. The 3–6 month windows in the timeline table are GitHub's estimates, not commitments.

13 / HANDOFF

GitHub's roadmap is not chasing a new pipeline syntax — it is making the existing YAML model's defaults trustworthy. Pair this with **the Jenkins roadmap** to understand the maturing self-hosted alternative, then continue to **CI/CD Decision Guide** for the side-by-side framework you can apply in your own org.

🔪 **EXAM** The attack surface in one breath: 2025–26 incidents (changed-files compromise, Nx toolchain injection, trivy tag poisoning, prompt-injected caches) all ran **trusted dependency** → **CI execution** → **secret exfiltration**, and the roadmap answers with four pillars — **lock dependencies** (SHA + hash, transitive closure), **scoped secrets** (per-secret binding, secrets: inherit deprecated), **execution protections** (rulesets gating actor + event against comment-and-control), **observe and contain** (data stream to SIEM plus L7 egress firewall, Monitoring before Enforcement) — adopting protections plus Immutable Releases first and the firewall last.

▶ Next: **The Choice**

Threads: **Approval Gates**

The Choice

Neither roadmap is chasing a new pipeline paradigm. Both consolidate around what already works — but they optimize for *different futures*. *Jenkins* is paying down debt to keep an existing install base healthy; *GitHub Actions* is locking down trust because it already won the adoption battle.

01 / OPTIMIZATION VECTORS

JENKINS

Survival & modernization

Close UX/infra gaps on a mature, volunteer-driven install base

vs

GITHUB ACTIONS

Trust & safety at scale

Make the simple YAML model secure by default at massive scale

ONE SENTENCE Jenkins reads like a deliberate open-source project *managing technical debt* (dark theme, Plugin Manager, Puppet→Ansible, JDK/Windows upgrades). Actions reads like a platform vendor *responding to real attacks* (dependency locking, scoped secrets, egress firewall). Neither is reinventing pipeline authoring — both are making the existing authoring safer and more observable.

02 / GOVERNANCE CONTRAST

Dimension	Jenkins	GitHub Actions
Governance	Community-driven, volunteer-dependent (CD Foundation); no committed dates	Vendor-driven (Microsoft/GitHub); estimated windows, still no hard SLAs
Headline 2026	Incremental UX + infra overhaul	Supply-chain security hardening
Cadence	Weekly · LTS every 4 weeks · advisories as needed	Continuous rollout (betas/GAs via Changelog)
Architecture change	None planned — controller/agent, Groovy DSL stays	None planned — YAML + new dependencies: lockfile

Threads: [Controllers & Agents](#)

Side-by-Side

03 / THE TWO ROADMAPS — FULL DIMENSION TABLE

Dimension	Jenkins	GitHub Actions
Biggest near-term unlock	Plugin Manager UI (exp.), UI/UX modernization, Puppet→Ansible	Arm64 GA, Immutable Releases GA, Execution Protections (rulesets)
Biggest risk addressed	Aging UI/plugin ecosystem, infra debt (Puppet, expiring MSI cert)	Supply-chain compromise of third-party Actions
Extensibility	~1,800 plugins — huge, uneven maintenance	~20k Marketplace Actions — huge, same trust problem being solved
Hosting	Self-hosted only (no official SaaS)	GitHub-hosted (SaaS) + self-hosted runners
AI / agentic	Not a current roadmap theme	Active — Copilot agents + security controls to contain them
Release model	Train (weekly/LTS)	Platform rollout (beta → GA)
Plugin/Action provenance	Manager UX revamp	Dependency locking + Immutable Releases + attestations
Security posture	Hardening via Script Security, JCaSC	Hardening via scoped secrets, egress firewall

04 / TIMELINE REALITIES

Jenkins horizons – read as priorities, not quarters

Released

- Dark theme, tables→divs, JEP-229, Temurin, multi-arch images

Preview

- Plugin Manager UI (2.552+), Pipeline-as-YAML, Jenkinsfile Runner

Current

- Look & Feel, Script Security approvals, CloudEvents, Puppet→Ansible

Near term

- A11y, JDK 25, Win2025, LF MSI cert (May 2026 expiry)

Future

- Full UI rework, JCaSC pluggable config, GraalVM/Quarkus

GitHub Actions windows – verify via Changelog (no hard SLAs)

Preview shipped

- Workflow Execution Protections (Jun 18 2026)

GA shipped

- Arm64 runners, Immutable Releases

Preview 3–6 mo

- Dependency locking, scoped secrets, data stream

Preview 6–9 mo

- Native egress firewall (monitoring → enforcement)

BOTH ARE DIRECTIONAL

Jenkins warns that Near/Future depend on contributor bandwidth; GitHub publishes "3–6 months" estimates, not commitments. Treat every non-GA date as a *sequencing hint* and re-verify against `roadmap.yml` and the GitHub Changelog before you plan migrations around them.

Threads: [Controllers & Agents](#) · [Artifact Digest](#) · [Approval Gates](#)

Decision Framework

05 / CHOOSE JENKINS WHEN

- You need **on-prem / air-gapped** execution (regulated, government, no egress).
- Build/deploy logic is highly **custom, heterogeneous, or legacy** — multiple SCMs, mixed Windows/Linux/mainframe, non-GitHub source.
- You have deep **Jenkins/Groovy expertise** and migration cost outweighs benefit.
- You need **very fine-grained control** over execution engine, distributed builds, custom scheduling.
- Cost model must be **zero marginal cost per build minute** (owned hardware).

06 / CHOOSE GITHUB ACTIONS WHEN

- Source already lives on **GitHub** — single strongest signal; PR/Issues/Checks integration is hard to replicate.
- You want **low-maintenance SaaS CI** with minimal infra ownership.
- You value a large, fresh **Marketplace ecosystem** and fast onboarding (YAML simpler than Groovy for most teams).
- You want the **newest supply-chain security defaults** (locking, scoped secrets, egress firewall) as they roll out in 2026.
- You are building **agentic / Copilot-assisted** workflows and want native platform support.

07 / CONSIDER BOTH TOGETHER (COMMON IN LARGER ORGS)

MIGRATING — Run legacy jobs on Jenkins while new services onboard to Actions. **SPECIALIZED** — Jenkins for on-prem workloads (hardware-in-the-loop, mainframe) while Actions handles standard app CI. **ORCHESTRATED** — Actions as the trigger/orchestration layer (PR gating, release tagging) calling Jenkins as a specialized executor via webhook/API.

```
# hybrid topology (conceptual)
PR / push → GitHub Actions (lint, test, gate) → if specialized →
  webhook → Jenkins controller (hardware-in-the-loop, mainframe, air-gapped build)
    \ else → Actions runners (build, SBOM, deploy to cloud)
# two systems, one lifecycle: SOURCE → PIPELINE → VALIDATION → ARTIFACT → RELEASE
```

08 / GUT-CHECK TABLE

If your priority is...	Lean toward
Regulatory / air-gapped compliance	Jenkins
Fastest time-to-first-pipeline	GitHub Actions
Total control over build infra	Jenkins
Minimal ops overhead	GitHub Actions
Legacy toolchain (SVN, Perforce, mainframe)	Jenkins
Native GitHub PR/Issue/Copilot integration	GitHub Actions
Zero per-minute cost at high volume (owned hardware)	Jenkins
Best-in-class supply-chain security defaults (2026)	GitHub Actions
Absolute plugin count (breadth, older)	Jenkins (~1,800)
Freshest, best-maintained integrations	GitHub Actions (~20k Marketplace)

Threads: [Controllers & Agents](#) · [Artifact Digest](#)

Tool vs Tool — The Whole Field

Four tools, one decision: match team size, hosting, and audit needs — not hype. Each cell distills its book chapter.

Dimension	Jenkins	Actions	GitLab	ArgoCD
Config lives	Jenkinsfile in repo, Groovy	Workflows in .github/workflows	Single .gitlab-ci.yml in repo	Manifests in ops git repo
Runners	Controller plus self-hosted agents	Hosted fleet plus ARC scale	Shared fleet plus tags	Agents pull inside cluster
Gates	Stages, input steps, approvals	Needs DAG, environments, reviewers	Rules, review apps, merge trains	Manual sync, waves, hooks
Secrets	Credentials store, scopes, bindings	Scoped secrets, OIDC, environments	Masked variables, environments	External operators, refs only
Cost shape	Zero per-minute, own hardware	Per-minute SaaS, free tier	Per-seat platform, shared runners	Cluster cost, free controller
Best fit	Regulated fleets, custom estates	GitHub SaaS teams, fast start	Single-platform shops, review apps	K8s fleets needing pull audit

WARNING No best tool — only best fit for team size, hosting, and audit needs.

EXAM Solo dev → Actions. Startup SaaS → Actions. Regulated bank → Jenkins or self-managed GitLab. K8s platform team → ArgoCD pull.

Next: [Convergence](#)

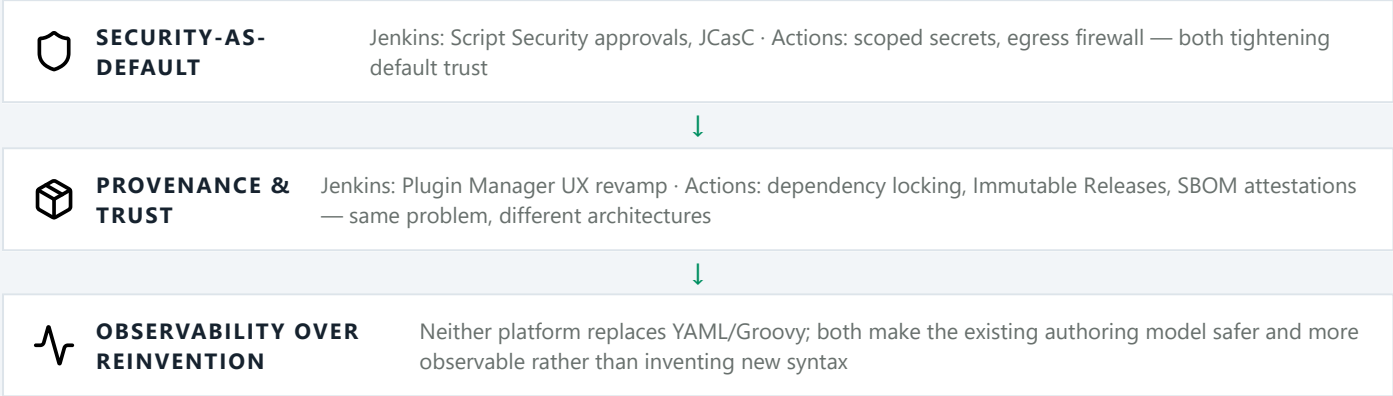
Threads: [Controllers & Agents](#) · [GitOps Pull Model](#)

Prefer questions? Take the [5-question tool picker](#) — then compare here.

Convergence

09 / WHERE THEY CONVERGE

Three shared directions — regardless of which platform you pick today, expect the ecosystem to keep tightening along these axes through 2026–27:



THE TAKEAWAY

If you choose Jenkins, budget for steady modernization (UI, infra, JDK) and harden Script Security now. If you choose Actions, enroll early in Execution Protections and Immutable Releases, and plan a staged rollout for scoped secrets + egress firewall. If you run both, use Actions to *gate* and Jenkins to *execute* the jobs only Jenkins can run.

10 / SUGGESTED ADOPTION SEQUENCE

Quarter (illustrative)	Jenkins track	Actions track
Now	LTS upgrades; qualify Java 21 mandatory; adopt Script Security granular approvals	Enable Immutable Releases (org); enable Execution Protections preview; audit secrets: <code>inherit</code>
Next	Watch Plugin Manager UI deltas; qualify JDK 25 + Win2025	Enroll dependency locking + scoped secrets previews; start data-stream SIEM plumbing
Later	Evaluate Pipeline-as-YAML (Preview) for new repos	Run egress firewall in Monitoring , validate allow-list, then Enforcement

Sequencing is guidance only — calendar it against your own risk + change windows, not against the roadmap's directional estimates.

Threads: [Artifact Digest](#) · [Approval Gates](#) · [Supply-Chain Security](#)

Decision Risks & Handoff

11 / FAILURE MODES — DECISION EDITION

MIGRATING FOR THE ROADMAP INSTEAD OF YOUR CONSTRAINTS

The most expensive mistake: moving a working Jenkins estate to Actions (or vice versa) because a roadmap looks shinier, without scoring your actual constraints — air-gap, legacy SCM, secret posture, team skill, cost at volume. Score the gut-check table first; only migrate where the signal is clear.

- **Treating Marketplace/plugin counts as quality** — ~1,800 Jenkins plugins and ~20k Actions both include unmaintained entries. Evaluate *specific* plugins/Actions for ownership, provenance, and update freshness — especially the ones that will touch secrets or deploys.
- **Adopting egress firewall in Enforcement without a monitoring phase** — you will deny real registries. Monitoring first is not optional.
- **Leaving `if: actor != '...'` sprawl instead of rulesets** — central policy is the durable fix for Comment-and-Control; per-workflow `if: guards` rot.
- **Assuming "Pipeline as YAML" means Groovy is deprecated** — it's additive and still in Preview; don't gate adoption on a horizon labeled Future.

Q4 We already run Jenkins well — should we add Actions?

Only if you score high on the Actions signals (code on GitHub, SaaS desire, supply-chain defaults, agentic work) *or* need a hybrid split (Actions gates standard app CI, Jenkins keeps specialized builds). Otherwise, invest in the Jenkins modernization path — LTS hygiene, JCasC, Script Security.

12 / CHECKLIST — BEFORE YOU CHOOSE

SCORE Fill the gut-check table with your stakeholders; require agreement on 3+ rows before choosing. **PROVE** — Run a one-repo pilot on the non-incumbent platform before org-wide commitment. **VERIFY** — Check live roadmaps (`roadmap.yml` + Changelog) the week you decide — not this PDF's snapshot. **HYBRID GATE** — If both, define the boundary contract early: which system gates, which executes, and how provenance flows across the boundary.

Threads: [Approval Gates](#) · [Supply-Chain Security](#)

Decision Risks & Handoff • continued

13 / SOURCES & HOW TO STAY CURRENT

```
# Jenkins (live)
roadmap.yml → github.com/jenkins-infra/jenkins.io/content/_data/roadmap/roadmap.yml
archive.yml → ../roadmap/archive.yml # completed/withdrawn
JEP-14      → github.com/jenkinsci/jep/tree/master/jep/14
# GitHub Actions (live)
roadmap     → github.com/github/roadmap # label:actions
changelog   → github.blog/changelog # ship dates
feedback    → GitHub Public Feedback Discussions
```

14 / HANDOFF

The roadmaps close the "strategy" arc — you now hold the 2026 context for both platforms and a framework for choosing between them. For implementation, return to the preceding chapters: **the platform roadmaps** for platform detail, or jump to the **the Labs Handbook** to build and gate a real pipeline. The decision is yours; the verification is continuous.

 **EXAM** The choice in one breath: **Jenkins modernizes to survive** (UX, plugin trust, infra debt on volunteer bandwidth) while **Actions hardens trust at scale** (locking, scoped secrets, firewall after real attacks) — choose Jenkins for air-gap, legacy SCM, total control and zero per-minute cost, Actions for GitHub-native SaaS, Marketplace breadth, 2026 supply-chain defaults and agentic work, and run **both** (Actions gates, Jenkins executes the exotic) whenever one estate cannot cover every constraint; score the gut-check, pilot the non-incumbent, verify live roadmaps the week you decide.

Kubernetes for Pipelines

Kubernetes is where pipelines run and where releases land. Same ideas as everywhere — runners become pods, deploys become desired state.

01 / WHERE KUBERNETES FITS

- **Agents on pods:** Jenkins agents and ARC runners scale as pods — ephemeral, labeled, bin-packed like any workload.
- **Delivery target:** ArgoCD syncs git state into clusters with waves and hooks — the pull-model gate from [ArgoCD & GitOps](#).
- **Preview envs:** ApplicationSets spin one env per branch — review apps without a separate platform.

WHEN TO USE KUBERNETES use when fleets share clusters and pull audit matters · use when elastic runners beat static VMs · use when preview envs multiply · avoid when a single service ships without ops · avoid when clusters cost more than the problem.

Tool vs Tool — The Whole Field

WARNING Kubernetes never fixes a broken pipeline — it scales whatever discipline already exists, good or bad.

🎓 **EXAM** Runner question → pods. Ordering question → sync waves. Audit question → git revert, not kubectl.

🔗 Next: [Quick Revision — Terms Table](#)



Quick Revision — Terms Table

Workflow	Pipeline YAML. Triggered by on: push, PR, schedule, dispatch.
Runner	Execution host. Hosted ubuntu-latest, self-hosted, or ARC scale-to-zero pools.
needs:	Sequences jobs into gates. Without it, jobs run in parallel.
environment:	Named target (staging/production). Required reviewers = approval gate.
workflow_call	Reusable workflow: one pipeline definition serving N repos.
Matrix	Fan-out of versions/platforms from one job block. Watch cost.
Merge train	Serializes merges onto latest green main with re-verification.
Review apps	Per-merge-request live environments, destroyed on merge via on_stop:.
rules:	Decides when GitLab jobs run. Replaces older only/except. Mis-scope = silent skip.

artifacts: / cache:	artifacts: passes files between jobs; cache: speeds reruns, lockfile-keyed.
Application	ArgoCD object: what (repo+path+revision) goes where (cluster+namespace).
Sync waves	Declarative ordering of resources: CRDs before CRs, migrations before app.
ApplicationSet	One template generating N Applications across envs and tenants.
Immutable Releases	Org-level guard making tag retargeting detectable. Closes tag poisoning.
Dependency locking	SHA + hash lockfile for Actions, transitive closure included. Deterministic runs.
Scoped secrets	Per-secret explicit binding to repo, branch, environment or trusted workflow.
Execution Protections	Rulesets governing actor + event. Shipped June 18, 2026. Kills comment-and-control.
Egress firewall	Native L7 firewall outside the runner VM. Monitoring first, Enforcement after.

KEY IDEA Cover the right column and define each term aloud. Any stumble goes back to its chapter.

Threads: [GitOps Pull Model](#) · [Approval Gates](#) · [Controllers & Agents](#)



Concept Questions — Distilled Answers

Q1 needs: vs parallel in Actions?

Jobs run parallel by default; needs: sequences them into gates.

Q2 Push vs pull deployment?

Push: external pipeline holds cluster creds. Pull: in-cluster agent syncs from git, creds stay inside.

Q3 Why does git revert beat redeploy for GitOps rollback?

Revert is reviewable, revertable and auditable — history stays the source of truth.

Q4 Which race do merge trains eliminate?

Green-on-branch-red-on-main: every merge re-verified against latest green main.

Q5 Which roadmap pillar first?

Workflow Execution Protections (in preview) plus Immutable Releases today. Firewall last.

Q6 Monitoring before Enforcement — why?

Enforcement without a validated allow-list denies real registries and endpoints.

Q7 Pipeline as YAML vs Groovy?

YAML is additive Preview for teams preferring it. Groovy Jenkinsfile stays the production path.

Q8 Strongest single signal for Actions?

Source already on GitHub — PR, Issues and Checks integration is hard to replicate.

Threads: [Controllers & Agents](#) · [Rollback & Recovery](#) · [GitOps Pull Model](#)

Command Cheat Sheet

Actions YAML

<code>on: [pull_request]</code>	CI on every PR
<code>jobs: { build: { runs-on: ubuntu-latest } }</code>	hosted runner job
<code>needs: [test, scan]</code>	gate deploy on verification
<code>environment: production</code>	approval-gated environment
<code>concurrency: { cancel-in-progress: true }</code>	cancel stale runs
<code>secrets: { NPM_TOKEN: \${ secrets.NPM_TOKEN } }</code>	explicit per-secret binding

GitLab + GitOps

<code>stages: [build, test, deploy]</code>	ordered stages, parallel jobs inside
<code>needs: [build]</code>	DAG edge skipping stage waits
<code>rules: [{ if: \$CI_MERGE_REQUEST_ID }]</code>	MR-only jobs
<code>environment: { name: review/\$CI_COMMIT_REF_NAME, on_stop: stop_review }</code>	review app lifecycle
<code>source: { repoURL, targetRevision: main, path }</code>	ArgoCD Application source
<code>syncPolicy: { automated: { prune: true, selfHeal: true } }</code>	drift healing with break-glass noted

NOTE Adoption order: Execution Protections plus Immutable Releases now, scoped secrets and locking in preview, egress firewall only after a monitoring window.

Threads: [Approval Gates](#) · [Controllers & Agents](#) · [GitOps Pull Model](#)

Self-Quiz + Answer Key

Q1 Hosted runners or self-hosted — the cost line?

Q2 Four Actions cost killers?

Q3 GitLab runners: shared vs tagged?

Q4 Promotion in GitOps?

Q5 Jenkins horizon labels in order?

Q6 Top-level SHA pinning: sufficient?

Q7 Comment-and-control countermeasure?

Q8 Hybrid split that works?

▪ Answer key

1. Public free; private pool first, then per-minute with macOS at 10x Linux.
2. macOS builds, no cache, sprawling matrices, failed jobs billing like green ones.
3. Shared fleet by default; tags route GPU, private-net and compliance jobs.
4. Pull request between env branches or folders — reviewable, revertable, auditable.
5. Released, Preview, Current, Near Term, Future — priorities, not dates.
6. No — transitive composite Actions stay open until dependency locking ships.
7. Workflow Execution Protections: actor + event rulesets at org level.
8. Actions gates standard app CI; Jenkins executes air-gapped and exotic builds.

FINAL CHECK Eight for eight from memory, twice in a row — the platform picture is yours. Return to the Labs Handbook and build.

Threads: [GitOps Pull Model](#) · [Controllers & Agents](#)